

Drzewa (struktury drzewiaste)

Kombinatoryka:

Drzewo to graf nieskierowany spójny, bez cykli.

(istnieje kilka równoważnych definicji)

Algorytmika:

Drzewo to struktura danych służąca do reprezentacji struktur hierarchicznych, np.:

- katalogów i plików systemu operacyjnego,
- dokumentu (rozdziały, sekcje, paragrafy etc.),
- wyrażenia algebraicznego,
- menu aplikacji/telefonu/serwisu,
- itd., itp.

Definicja indukcyjna drzewa (D.Knuth):

Drzewo, nad uniwersum wierzchołków (węzłów) U , to niepusty skończony zbiór $T \subseteq U$, w którym

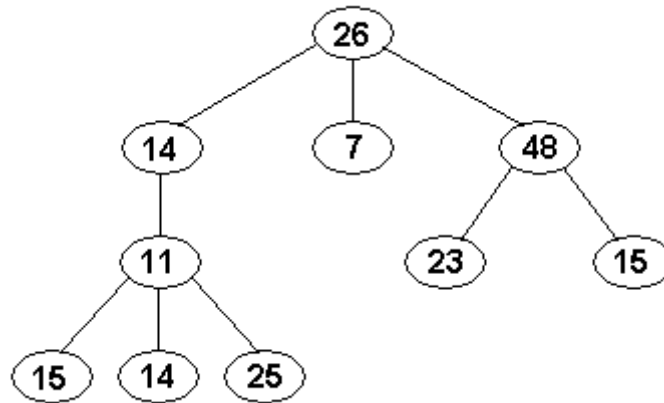
- istnieje wyróżniony wierzchołek $x \in T$ zwany korzeniem,
- pozostałe wierzchołki są podzielone na $k \geq 0$ parami rozłącznych podzbiorów $T_1, \dots, T_k$, będących drzewami, i nazywanych poddrzewami (korzenia) drzewa T .

W myśl tej definicji:

- zbiór pusty nie jest drzewem,
- zbiór jednoelementowy jest drzewem złożonym z samego korzenia (baza indukcji w definicji),
- poddrzewa danego wierzchołka są uporządkowane (ponumerowane).

Terminologia:

- elementy zbioru T : węzły (wierzchołki)
- wyróżniony element $x \in T$: korzeń
- podzbiór T_i , $i=1, \dots, k$: poddrzewo (poddrzewo ma swój korzeń, który z kolei ma swoje poddrzewa itd., zatem każdy węzeł jest korzeniem drzewa lub pewnego poddrzewa)
- jeśli x jest korzeniem drzewa T , y korzeniem poddrzewa T_i , to y jest następnikiem (dzieckiem) dla x , x jest poprzednikiem (rodzicem) dla y ; analogicznie, rozciągając te relacje, mówimy o przodkach i potomkach w drzewie (jak w drzewie genealogicznym)
- zbiór dwuelementowy $\{x, y\}$, gdzie x jest poprzednikiem dla y : krawędź drzewa łącząca x i y
- ciąg parami różnych wierzchołków taki, że każde dwa wierzchołki sąsiednie w tym ciągu są połączone krawędzią: ścieżka w drzewie
- liczba krawędzi na ścieżce: długość ścieżki (czyli liczba węzłów na ścieżce minus 1)
- liczba poddrzew w węźle x : stopień węzła x
- węzeł o stopniu równym zero: liść
- długość najdłuższej ścieżki od korzenia do liścia: wysokość drzewa
- długość ścieżki od węzła v do korzenia drzewa: głębokość węzła v w drzewie
- skończony ciąg drzew: las (las może być pusty)



- korzeń: 26
- stopień korzenia: 3
- korzenie poddrzew (korzenia): 14, 7, 48
- następniki węzła 48: 23, 15
- stopień węzła 48: 2
- poprzednik węzła 25: 11
- wysokość drzewa: 3
- głębokość węzła 11: 2
- liczba węzłów: $n=10$
- liście: 15, 14, 25, 7, 23, 15

Inna popularna nazwa struktury danych "drzewo":

drzewo wieloarne,

w odróżnieniu od drzewa binarnego omawianego poniżej.

Drzewa binarne

Nieformalnie: drzewo binarne to drzewo, w którym:

- każdy wierzchołek ma stopień nie większy niż 2,
- każdy następnik jest albo lewym albo prawym następnikiem, co najwyżej jednym takim,
- drzewo binarne może być puste.

Formalna definicja drzewa binarnego (D.Knuth)

Ustalono jest uniwersum elementów U .

- zbiór pusty jest drzewem binarnym (pustym) nad uniwersum U ;
- jeśli T_1, T_2 są drzewami binarnymi nad uniwersum U , $r \in U$, to trójka $T = (r, T_1, T_2)$ jest drzewem binarnym nad U , przy czym:
 - r jest korzeniem drzewa T ,
 - T_1 jest lewym poddrzewem drzewa T ,
 - T_2 jest prawym poddrzewem drzewa T .

UWAGA:

FORMALNIE, DRZEWO BINARNE NIE JEST DRZEWEK.

To po prostu pewna prosta struktura algebraiczna, łatwo reprezentowalna jako struktura danych.

Ale, wprowadzone dla drzew pojęcia:

- poprzednika, następnika, poddrzewa,
- liścia i węzła wewnętrznego,
- ścieżki, głębokości wierzchołka, stopnia wierzchołka, itd.

intuicyjnie przenoszą się na drzewa binarne.

Własności kombinatoryczne drzew binarnych:

(mówiąc „drzewo” mamy na myśli „drzewo binarne”, aż do odwołania)

1. Maksymalna możliwa liczba wierzchołków na głębokości d wynosi 2^d .
2. Maksymalna moc drzewa (liczba wierzchołków) o wysokości h wynosi $2^{h+1}-1$, minimalna wynosi $h+1$.
3. Wysokość h drzewa o n wierzchołkach spełnia nierówności $\lfloor \lg n \rfloor \leq h \leq n-1$.
4. Liczba pustych poddrzew (tzw. liści zewnętrznych) w drzewie o n wierzchołkach wynosi $n+1$.
5. Jeśli n_d oznacza liczbę wierzchołków stopnia d to

$$n_0 = n_2 + 1$$

Własności 4, 5 – dowód indukcją na budowę (definicję) drzewa binarnego.

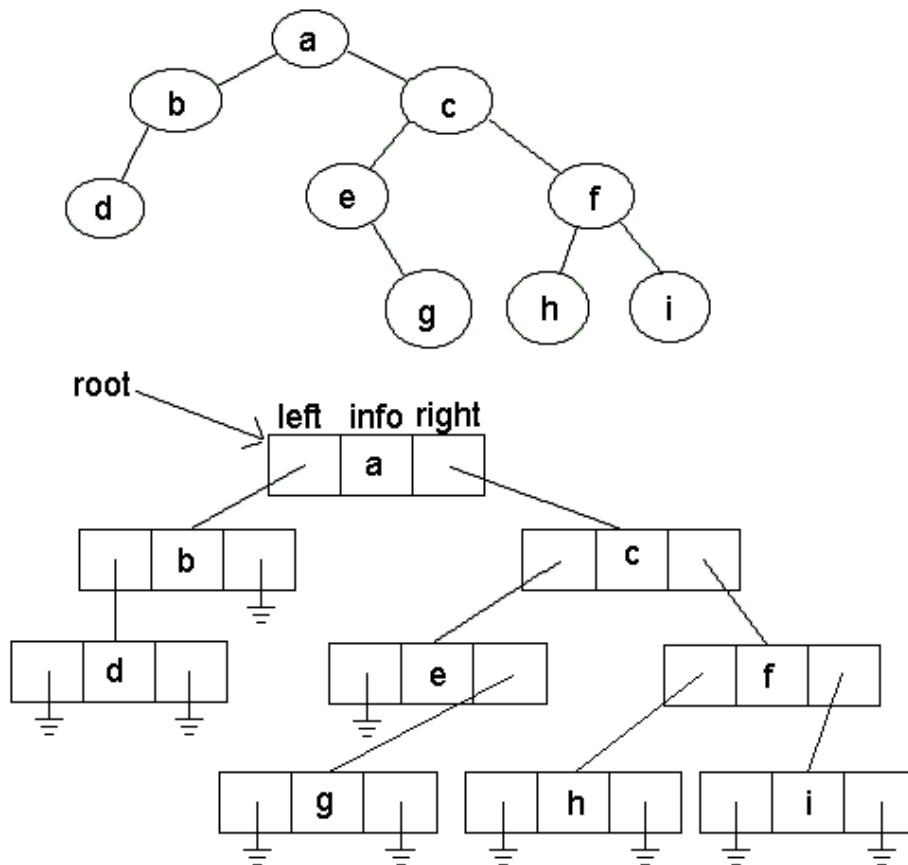
Reprezentacja wskaźnikowa drzewa binarnego:

Węzeł: struktura zawierająca:

- pole 'info' (element uniwersum U , czyli ustalonego typu), lub 'key',
- wskaźnik 'left' do korzenia lewego poddrzewa (równy NULL jeśli lewe poddrzewo puste),
- wskaźnik 'right' do korzenia prawego poddrzewa (NULL jeśli puste),
- (opcjonalnie) wskaźnik 'up' do poprzednika (up=NULL w korzeniu).

Drzewo zadane jest przez

- wskaźnik 'root' do korzenia,
- root = NULL gdy drzewo puste.



Przegląd drzewa binarnego

Dane: wskaźnik root do korzenia drzewa binarnego.

Cel: zbadać wszystkie węzły drzewa o korzeniu root.

Algorytm przeglądu jest szkieletem wielu algorytmów przetwarzających informację w strukturach drzew.

Przykłady procedury "badanie węzła":

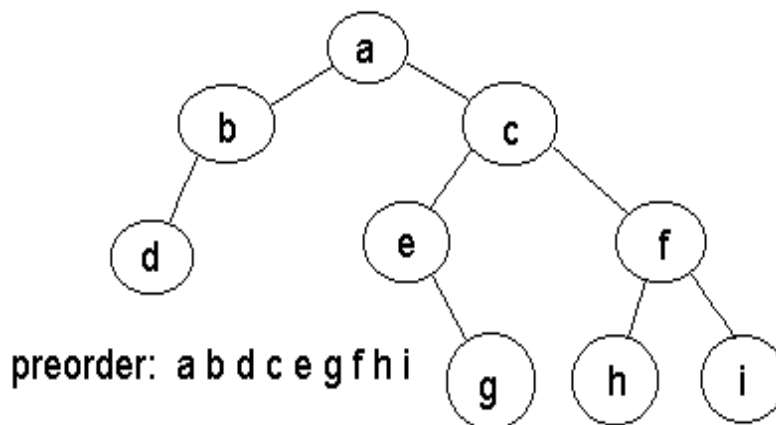
- wypisanie zawartości pola info;
- sprawdzenie czy jest to wartość szukana;
- numerowanie węzłów;
- obliczanie funkcji zależnej od węzła oraz od wartości poprzednika i/lub następników (obliczanie wyrażenia) .

Trzy podstawowe porządki przeglądu drzewa binarnego:

1. Porządek zstępujący (prefiksowy, "preorder")

Jeśli drzewo jest puste to nic nie rób, w przeciwnym przypadku:

1. zbadaj korzeń;
2. (rekurencyjnie) przeglądaj lewe poddrzewo w porządku zstępującym;
3. (rekurencyjnie) przeglądaj prawe poddrzewo w porządku zstępującym.



Pseudokod algorytmu przeglądu preorder, wprost z definicji:

```
procedure preorder (p : wskaźnik)
    if p = NULL then return;
    zbadaj(p);           // np. wypisz info(p)
    preorder(left(p));
    preorder(right(p));
end.

main:
    preorder(root);
```

Praktycznie nieco szybciej może działać wersja, w której zakładamy, że argument funkcji jest niepusty - wtedy zmniejsza się liczba wywołań funkcji.

```
procedure preorder (p : wskaźnik)
    zbadaj(p) ;
    if left(p) ≠ NULL then preorder(left(p)) ;
    if right(p) ≠ NULL then preorder(right(p)) ;
end.
```

```
main:
    if root ≠ NULL then preorder(root) ;
```

Ale w sumie jest to różnica głównie estetyczna.

Złożoność algorytmu przeglądu drzewa: $\Theta(n)$.

Dokładniej (dla wersji pierwszej): liczba wywołań procedury preorder wynosi:

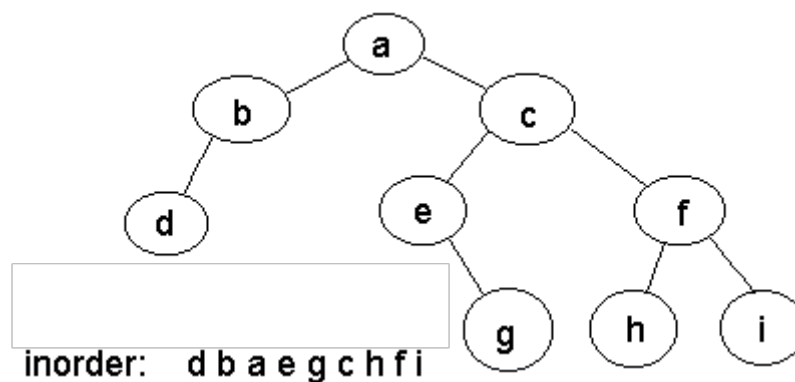
- raz dla każdego węzła, w sumie n wywołań,
- raz dla każdego pola równego NULL (czyli liścia zewnętrznego), $n+1$ takich wywołań;
- razem: $2n+1$ wywołań.

Każde wywołanie zajmuje stały czas.

2. Porządek symetryczny (infiksowy, "inorder")

Jeśli drzewo jest puste to nic nie rób, w przeciwnym przypadku:

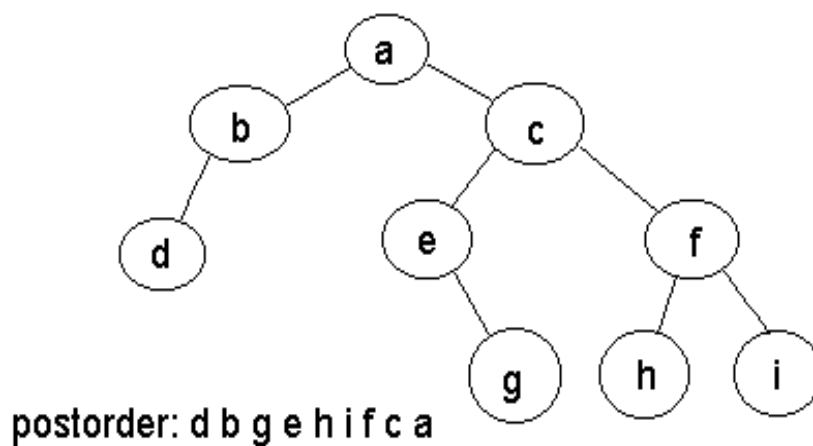
1. (rekurencyjnie) przeglądnij lewe poddrzewo w porządku symetrycznym;
2. zbadaj korzeń;
3. (rekurencyjnie) przeglądnij prawe poddrzewo w porządku symetrycznym.



3. Porządek wstępujący (postfiksowy, "postorder")

Jeśli drzewo jest puste to nic nie rób, w przeciwnym przypadku:

1. (rekurencyjnie) przeglądnij lewe poddrzewo w porządku wstępującym;
2. (rekurencyjnie) przeglądnij prawe poddrzewo w porządku wstępującym;
3. zbadaj korzeń.



Wersja nierekurencyjna algorytmu przeglądu

Istnieje elegancka wersja nierekurencyjna - iteracyjna ze stosem - dla wszystkich trzech porządków przeglądu drzewa binarnego. Przydatna w pewnych praktycznych sytuacjach.

Algorytm nierekurencyjny dla porządku inorder:

```

S := new Stack();    // S - stos początkowo pusty
p := root;           // p - wskaźnik bieżący
while p ≠ NULL or S.nonempty do
  if p ≠ NULL then
    S.push(p);
    p := left(p); // przejście do lewego poddrzewa
  else
    p := S.pop();
    zbadaj(p);
    p := right(p); //przejście do prawego poddrzewa
end;
  
```

Poprawność: indukcją na budowę drzewa, że algorytm:

- odwiedza węzły w odpowiedniej kolejności;
- nie zmienia zawartości stosu zastanej w momencie zejścia do poddrzewa.

Złożoność: jak poprzednio – liniowa.

Ćwiczenie:

Skonstruuj analogiczne algorytmy iteracyjne dla inorder i postorder.

Listowe reprezentacje drzewa

czyli jak zapamiętać drzewo zapisując ciąg jego wierzchołków (w tablicy).

Fakt 1.

Lista węzłów drzewa binarnego w którymkolwiek z porządków nie definiuje drzewa jednoznacznie.

Fakt 2.

Jeśli w drzewie binarnym

- klucze (info) są unikalne,
- LP jest listą kluczy w porządku preorder,
- LI jest listą kluczy w porządku inorder,

to para (LP, LI) definiuje jednoznacznie to drzewo.

Dowód:

algorytm konstrukcji reprezentacji wskaźnikowej drzewa dla zadanych list w preorder i inorder.

Jego poprawność: łatwą indukcją na budowę drzewa.

```

main:
    // tablice pre[1..n], in[1..n] są globalne
    root := BuildTree(1, 1, n);
end.

function BuildTree (i, j, k)
// zwraca wskaźnik do korzenia
// k = długość aktualnych podtablic
// buduj drzewo dla pre[i..i+k-1], in[j..j+k-1]
    if k = 0 then return NULL;
    p := new node;
    info(p) := pre[i];
    (*)znajdź r ∈ [j..j+k-1] t.ż. pre[i]=in[r];
    left(p) := BuildTree(i+1, j, r-j);
    right(p) := BuildTree(i+r-j+1, r+1, j+k-r);
    return p;
end.

```

Złożoność: zależy od realizacji instrukcji (*):

- przegląd od lewej do prawej: $\Theta(n^2)$
- przegląd z obu końców tablicy równocześnie:
 $\Theta(n \log n)$ *(jak to uzasadnić?)*

Fakt 3.

Lista postorder wraz z listą inorder definiują jednoznacznie drzewo.

Fakt 4.

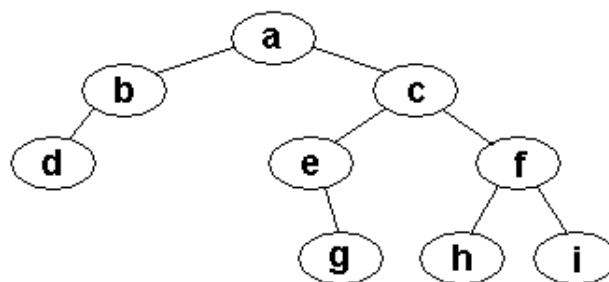
Lista preorder wraz z listą postorder nie definiują jednoznacznie drzewa.

Pokaż kontrprzykład.

Reprezentacja preorder ze znacznikami poddrzew.

Jest to, w kolejności preorder, lista trójek postaci $u=(info, L, R)$, gdzie

- 'info' jak w węźle,
- $L = 1$ jeśli węzeł u ma niepuste lewe poddrzewo, w przeciwnym przypadku 0,
- $R = 1$ jeśli węzeł u ma niepuste prawe poddrzewo, w przeciwnym przypadku 0.



```
info:  a b d c e g f h i
left:  1 1 0 1 0 0 1 0 0
right: 1 0 0 1 1 0 1 0 0
```

Fakt 5.

Reprezentacja preorder ze znacznikami jest jednoznaczna.

Dowód:

Na podstawie poniższego Faktu 6 o podobieństwie.

Definicja.

Drzewa binarne T, T' są podobne, jeśli

1. $T = T' = \emptyset$, lub
2. $T \neq \emptyset, T' \neq \emptyset$, oraz
 - drzewa $\text{left}(\text{root}(T))$ i $\text{left}(\text{root}(T'))$ są podobne,
 - drzewa $\text{right}(\text{root}(T))$ i $\text{right}(\text{root}(T'))$ są podobne.

Analogicznie definiujemy izomorfizm drzew, dodając warunek równości odpowiadających sobie węzłów ($\text{root}(T) = \text{root}(T')$).

Fakt 6.

Niech:

$u_1, \dots, u_n$ – węzły drzewa T w porządku preorder,

$u'_1, \dots, u'_{n'}$ – węzły drzewa T' w porządku preorder.

Wówczas T i T' są podobne wtedy i tylko wtedy gdy:

1. $n = n'$

2. $L(u_i) = L(u'_i), R(u_i) = R(u'_i), i=1, \dots, n$.

Dowód: w oparciu o definicje drzewa i podobieństwa drzew - ćwiczenie.

Fakt 7.

Analogiczna reprezentacja postorder definiuje drzewo jednoznacznie.

Fakt 8.

Analogiczna reprezentacja inorder ze znacznikami nie definiuje drzewa jednoznacznie.

Podaj kontrprzykład.

Wniosek:

poprawny jest poniższy algorytm kopiowania drzewa binarnego:

```

// WE: drzewo T, WY: T' = kopia T
if T=∅ then
    return NULL;
w := pusta struktura wierzchołka;
T' := (w, ∅, ∅);           // tylko korzeń
wykonaj synchroniczny przegląd drzew T, T' w preorder,
    stosując poniższą procedurę badania węzła:
visit(p,p'):               // p jest w T, p' jest w T'
    info(p') := info(p);
    if left(p) ≠ NULL then
        utwórz nowy węzeł, dowiaż go z lewej do p';
    if right(p) ≠ NULL then
        utwórz nowy węzeł, dowiaż go z prawej do p';
end visit;
return T';

```

Złożoność algorytmu kopiowania drzewa: $\Theta(n)$.

Uwagi:

- szkieletem algorytmu kopiowania jest jakikolwiek algorytm przeglądu drzewa w preorder,
- przeglądający oba drzewa równocześnie
- w pewnym (niekoniecznie jednym) miejscu tego algorytmu jest wywołanie procedury "zbadaj(v)",
- tę procedurę zastępujemy powyższą procedurą visit(p,p').